SQL
S3

EDC- 2026
Sh. Dowlatshahi

Course content

S1: Introduce the course, data concepts and the implementation of the software

S2: Create the database and its tables on the software

S3: Manipulate, retrieve and analyze data

S4: Establish the basic notions of using SQL

S5: Understand and practice jointures

S6: Settle the basics of SQL, Compare joins with subqueries

S7: Go further with the window functions

S8: Present the group work, Prepare for the exam

Session 3 :DML & SELECT

- SELECT with WHERE clause
- Perform simple queries Interpret the results
- Explaining the first assignment

UPDATE : Changing row values

ALTER: Changing table structure

Single Quotation: ' ' String/text values

Double Quotation: " " Column/table names (special/reserved)

SELECT

In PostgreSQL, the SELECT statement is used to retrieve data from a database table.

SELECT column1, column2
FROM table_name;

Customizing SELECT

1. SELECT Columns
2. SELECT with WHERE
3. SELECT with ORDER BY
4. SELECT with LIMIT
5. SELECT with Aggregation
6. SELECT with GROUP BY
7. SELECT with JOIN

etc,...

Filtering Records- WHERE (numeric and string values)

The WHERE clause in PostgreSQL filters records based on specified conditions, narrowing down results for SELECT, UPDATE, or DELETE operations. It works with various data types and supports comparison operators like =, >, <, >=, <=, and <>.

SELECT: what **columns** to display

WHERE: which **rows** to include

WHERE - books dataset

1. Numeric Example:

```
SELECT *
FROM books_goodreads
WHERE average_rating > 4.9;
```

2. String Example:

```
SELECT *
FROM books_goodreads
WHERE  publisher  = 'Broadway Books';
```

The WHERE clause helps you filter the rows you want to see. It’s used with SELECT to set conditions for the data you retrieve.

SELECT column1, column2
FROM table
WHERE (condition)

Question: Retrieve the book title and average rating for all books where the average rating is greater than 4.5.

Answer: Retrieve the book title and average rating for all books where the average rating is greater than 4.5.

```
SELECT title, average_rating
FROM books_goodreads
WHERE average_rating > 4.5;
```

Compound Where- AND

Question: Retrieve all books with an average rating greater than 4.1 and published by "Broadway Books."

Compound Where- AND

Answer: Retrieve all books with an average rating greater than 4.1 and published by "Broadway Books."

```
SELECT *
FROM books_goodreads
WHERE average_rating > 4.1 AND publisher = 'Broadway Books';
```

In SQL, dates need to be in single quotes (') so the system knows it's a date.
Without the quotes, it won't work because SQL thinks it's something else, like a column name.

Question: List all books that have been rated (excluding those with an average rating of 0).

Answer: List all books that have been rated (excluding those with an average rating of 0).

```
SELECT *
FROM books_goodreads
WHERE average_rating <>0
```

- `>` Greater than or after
- `<` Less than or before
- `=` Equal to
- `>=` Greater than or equal to
- `<=` Less than or equal to
- `<>` Not equal to

LIMIT

```
SELECT column1, column2
FROM table_name
LIMIT number_of_rows;
```

Question: Retrieve the titles of the top 5 books with an average rating greater than 4.5.

Answer: Retrieve the titles of the top 5 books with an average rating greater than 4.5.

```
SELECT title
FROM books_goodreads
WHERE average_rating > 4.5
LIMIT 5;
```

Order Of writing a Query

1. SELECT
2. FROM
3. WHERE
4. LIMIT

Order Of executing a Query

1. FROM
2. WHERE
3. SELECT
4. LIMIT

Why is it Different?

- Writing follows the order that’s easiest for humans to read.

- Execution follows an order that’s most efficient for the database to get the right result.

It’s like searching for something in a library:

1. You first look at the books in the right section (FROM),
2. Filter out irrelevant ones (WHERE),
3. Then pick the details you need (SELECT),
4. Finally limit the number of results (LIMIT).

OR operator

The OR operator lets you select rows where at least one condition is true. It’s used with WHERE to filter data.

OR - Invalid

```
SELECT *
FROM books_goodreads
WHERE average_rating= 4 OR 5;
```

```
ERROR:  argument of OR must be type boolean, not type integer
LINE 3: WHERE average_rating= 4 OR 5;
```

OR - Correct

```
SELECT *
FROM books_goodreads
WHERE average_rating= 4 OR average_rating=5;
```

AND Operator

If we want to satisfy all criteria in our filter, we need to use AND with WHERE.

AND - Invalid

```
SELECT *
FROM books_goodreads
WHERE average_rating > 4.5 AND < 5;
```

```
ERROR:  syntax error at or near "<"
LINE 3: WHERE average_rating > 4.5 AND < 5;
```

AND - Correct

```
SELECT *
FROM books_goodreads
WHERE average_rating > 4.5 AND average_rating < 5;
```

Combining AND & OR

When combining AND and OR, make sure to specify the field for each condition. Use **parentheses** to control the order of execution and ensure the query returns the correct results.

Question: Find books that have an average rating greater than 4.5 and were published after 2010, or have the publisher "Broadway Books".

Question: Find books that have an average rating greater than 4.5 and were published after 2010, or have the publisher "Broadway Books".

```
SELECT title, average_rating, publication_date, publisher
FROM books_goodreads
WHERE (average_rating > 4.5 AND publication_date > '2010-12-31')
OR publisher = 'Broadway Books';
```

Between- quick way to filter values within a range

Question: Your query should return the titles, average ratings, and publication dates of books with ratings between 4.5 and 5 (inclusive).

Between- quick way to filter values within a range

Answer: Your query should return the titles, average ratings, and publication dates of books with ratings between 4 and 5 (inclusive).

- SELECT title, average_rating, publication_date
FROM books_goodreads
WHERE average_rating >= 4.5 **AND** average_rating <= 5;

- SELECT title, average_rating, publication_date
FROM books_goodreads
WHERE average_rating **BETWEEN** 4.5 **AND** 5;

Question: write a query that returns the titles, publication dates, and publishers of books published between January 1, 2010, and December 31, 2020, by "Penguin Books".

Answer: Write a query that returns titles, publication dates, and publishers of books published between January 1, 2010, and December 31, 2020, by "Penguin Books".

```
SELECT title, publication_date, publisher
FROM books_goodreads
WHERE publication_date BETWEEN '2010-01-01' AND
'2020-12-31'
AND publisher = 'Penguin Books';
```

IN: If we want to filter based on multiple conditions, using OR can get messy. Instead, we can use IN to make it simpler.

Question: Write a SQL query to retrieve the title and publisher of all books whose publisher is either "Penguin" or "HarperCollins". (specific publishers)

IN

Answer: Write a SQL query to retrieve the title and publisher of all books whose publisher is either "Penguin" or "HarperCollins".

```
SELECT title, publisher
FROM books_goodreads
WHERE publisher IN ('Penguin', 'HarperCollins');
```

IN - Multiple conditions

These queries will return books with any of those ratings.

- SELECT title, average_rating
FROM books_goodreads
WHERE average_rating = 4.0
  OR average_rating = 4.5
  OR average_rating = 5.0;

- **SELECT title, average_rating**
FROM books_goodreads
WHERE average_rating IN (4.0, 4.5, 5.0);

We can use the WHERE clause to filter text, but sometimes we want to search for patterns instead of exact text. For this, we use **LIKE**, **NOT LIKE**, and **IN**.

LIKE: Finds records that match a pattern.

NOT LIKE: Finds records that do not match a pattern.

% : Matches many characters

Underscore (_): Matches one character.

These keywords allow us to filter texts more flexibly.

LIKE

Question: Find all books where the title contains "Java".

LIKE

Answer: Find all books where the title contains "Java":

```
SELECT *
FROM books_goodreads
WHERE title LIKE '%Java%';
```

NOT LIKE

Question: Find all books where the title does not contain "Python".

NOT LIKE

Answer: Find all books where the title does not contain "Python":

```
SELECT title
FROM books_goodreads
WHERE title NOT LIKE '%Python%';
```

% at the beginning:
LIKE '%Great' matches any string that ends with "Great", such as "A Great" or "The Great".

% at the end:
LIKE 'Great%' matches any string that starts with "Great", such as "Great Gatsby" or "Great Adventure".

Between %s:
LIKE '%Great%' matches any string that contains "Great" anywhere, such as "The Great Gatsby" or "Great Adventure".

Underscore

Matches exactly one character: Find all books whose title is 3 characters long and starts with "J":

Example:

```
SELECT title
FROM books_goodreads
WHERE title LIKE 'J__';
```

'Jan' OK
'J n' OK
'Jane' NOT OK
'J' NOT OK

Commonly used functions in WHERE clauses in PostgreSQL:

1. LOWER() / UPPER()
Used to make text comparisons case-insensitive.

```
SELECT * FROM books_goodreads
WHERE LOWER(publisher) = 'penguin';
```

Step1. Make it Lower
Step2. Compare it with 'penguin'

2. NOW() / CURRENT_DATE

Used to filter based on the current time or date.

```
SELECT * FROM books_goodreads
WHERE publication_date < CURRENT_DATE;
```

3. EXTRACT()
Pulls out parts of a date (year, month, etc.).

```
SELECT * FROM books_goodreads
WHERE EXTRACT(YEAR FROM publication_date) = 2020;
```

Description- Homework 2

This dataset contains information about Indian startups that received funding during the early days of the COVID-19 pandemic. It includes essential details such as startup names, industries, locations, funding amounts, investors, and funding rounds.

In this assignment, you'll practice using SQL to filter data. Focus on applying the SELECT and WHERE clauses to extract specific information based on conditions such as the month, investor, and location.

Tips

1. Set your keyboard to **English**.
2. Ensure your file is in **.csv** format.
3. The columns **order**, **names**, and **data types** in your file should be matched with your query.
4. After executing your query, you can run this query to view your data.

```
SELECT * FROM table_name LIMIT 10;
```